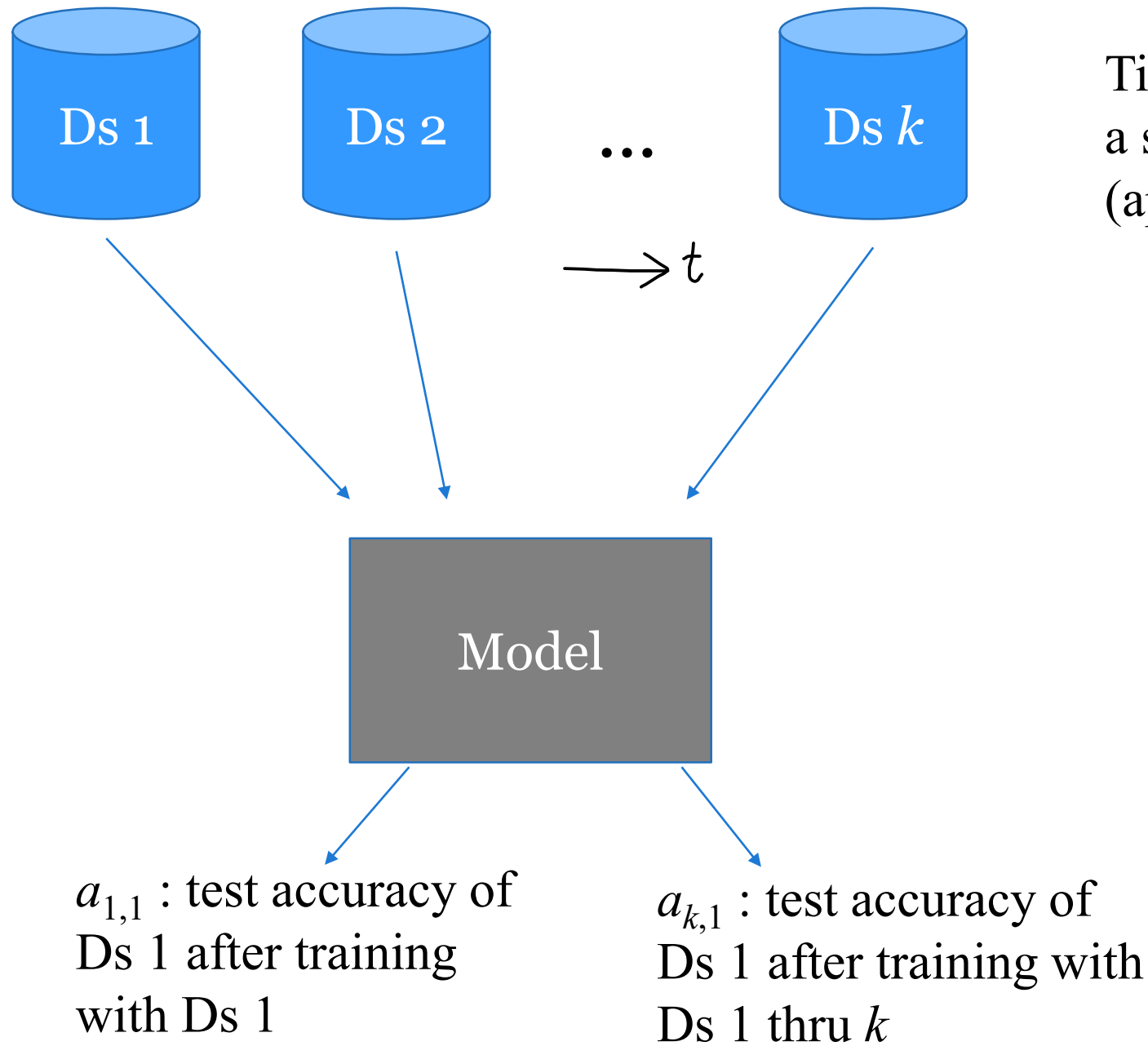


Continual Learning (CL) in ML/DL

Essential and increasingly critical technique for real-world projects in industry

12-4-24, 12-9-25

CJ Chung



Time dependent k datasets for
a specific domain
(application)

Continual Learning (CL)

- Continual learning is a subfield of ML that deals with incrementally training neural networks on continually arriving data.
- Because the neural networks are only optimized on the currently available data, they overwrite the old parameters (instead of retraining the model with merged data)
- In overwriting them, old knowledge *usually* is destroyed, i.e. *forgotten*.
- CL is needed for example, Self-drive Lane/Road keeping models

Continual Learning (CL)

- an **essential and increasingly critical technique in industry**, especially for real-world ML/DL systems,
 - because it allows models to adapt to constantly changing data and environments (like trends, user preferences, new info) *after* deployment,
 - preventing performance degradation and the need for expensive full retraining, making them more efficient, relevant, and robust over time.
- It solves the "catastrophic forgetting" problem, enabling systems like recommendation engines, fraud detectors, and chatbots to stay accurate and valuable.

What is few shot learning vs continual learning?

- In few-shot learning, the goal is to learn novel (new) concepts with as few samples as possible, i.e. evaluating the capability of adapting to new tasks.
- Whereas in continual learning(CL), the ability to learn an increasing amount of concepts while not forgetting old ones is evaluated.
- Our ultimate goal is CL with few-shot

The Metrics of Continual Learning

- Average accuracy: measures the test performance
- Backward transfer: measures catastrophic forgetting occurred or not
- Forward transfer: evaluates the effectiveness of continued training compared to task-specific training from scratch

<https://towardsdatascience.com/the-metrics-of-continual-learning-08f2d1cd959b>

Average Accuracy

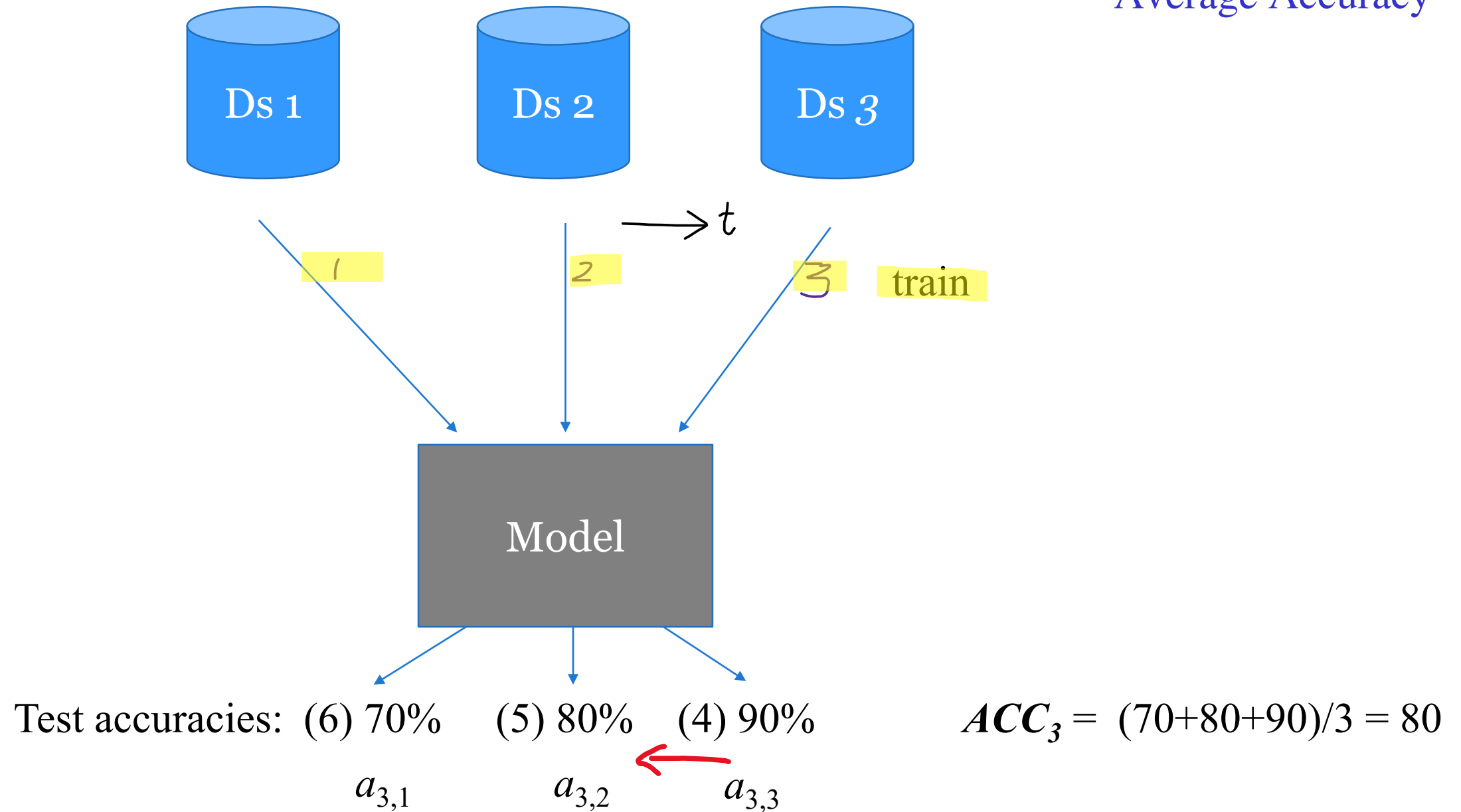
$$ACC_k = \frac{1}{k} \sum_{j=1}^k a_{k,j}$$

- it measures the (test-set) accuracy of each task, and then computes the average over the task specific accuracies
- k is the current task and $a_{k,j}$ denotes the test accuracy on the previous task j ($j \leq k$) after the training on task k .
- Example: Assume we are training a model on three tasks, 1, 2, 3. after training on task 3, we evaluate on tasks 1 to 3. In the last case, after training, the equation above will be:

$$ACC_k = \frac{1}{3} \sum_{j=1}^3 a_{3,j} = \frac{1}{3}(a_{3,1} + a_{3,2} + a_{3,3})$$

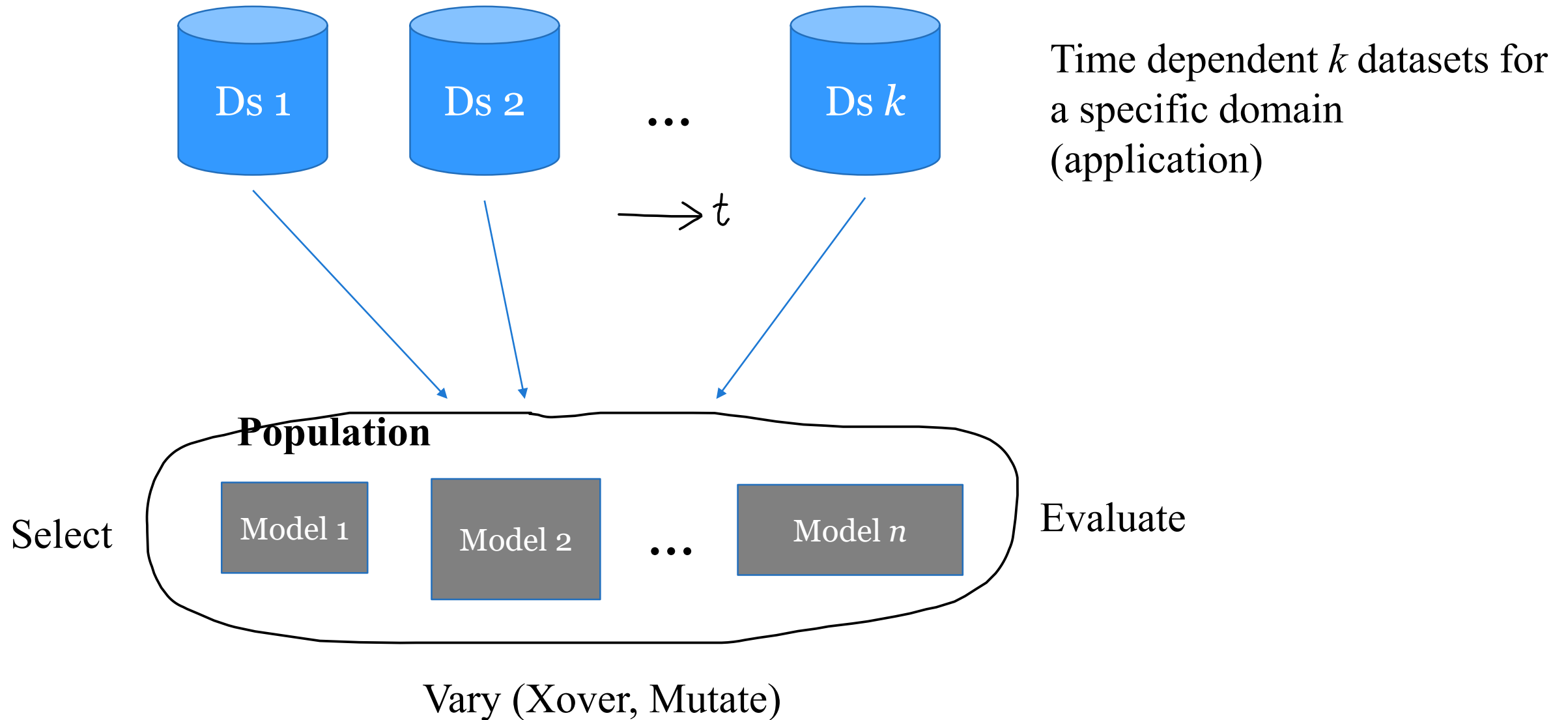
Test accuracy on task 1 after the training on task 1-3

Average Accuracy



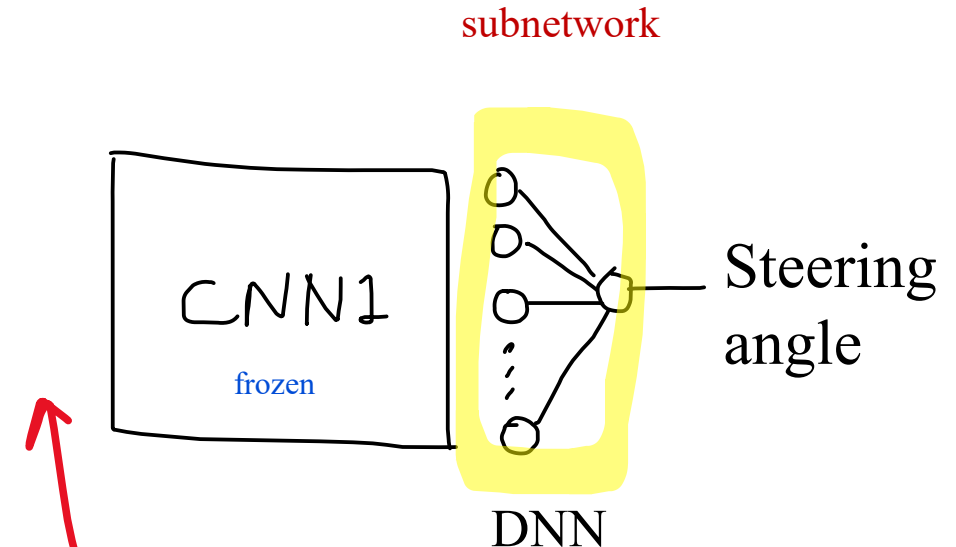
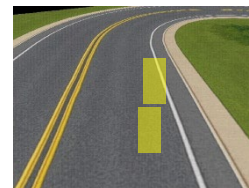
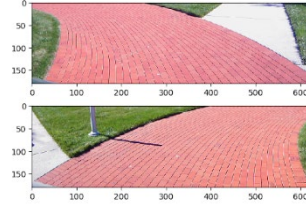
Applying EC to Continual Learning

- CL requires models to **acquire new knowledge without catastrophic forgetting**, adapt over time, and remain computationally efficient.
- EA provide the following mechanisms for CL's goals: population diversity, selection, mutation-based exploration, and architecture evolution.
 - Evolutionary Optimization of Connection Weight Subsets, Hyperparameters and Learning-Rules (details next slide)
 - Evolutionary Architecture Growth (NeuroEvolution for CL): evolving neural network weights + structures as tasks arrive.
 - Evolutionary *Dynamic* Selection of “Expert” Networks: Maintain a ***population*** of expert networks, each representing different specializations.
 - ...



Evolutionary Optimization of Connection Weight Subsets

- Instead of updating all weights by gradient descent when new data arrives, EAs can evolve **task-adaptive subnetworks**.
- When new data arrives, **evolve the subnetwork** responsible for that data, leaving old subnetworks unchanged.



Can we introduce subnet here?

High-Level Combined EC Algorithm for CL (Example)

1. Initialization

Train base network on initial task.

2. New Dataset Arrives: Local Evolutionary Search by Varying the models

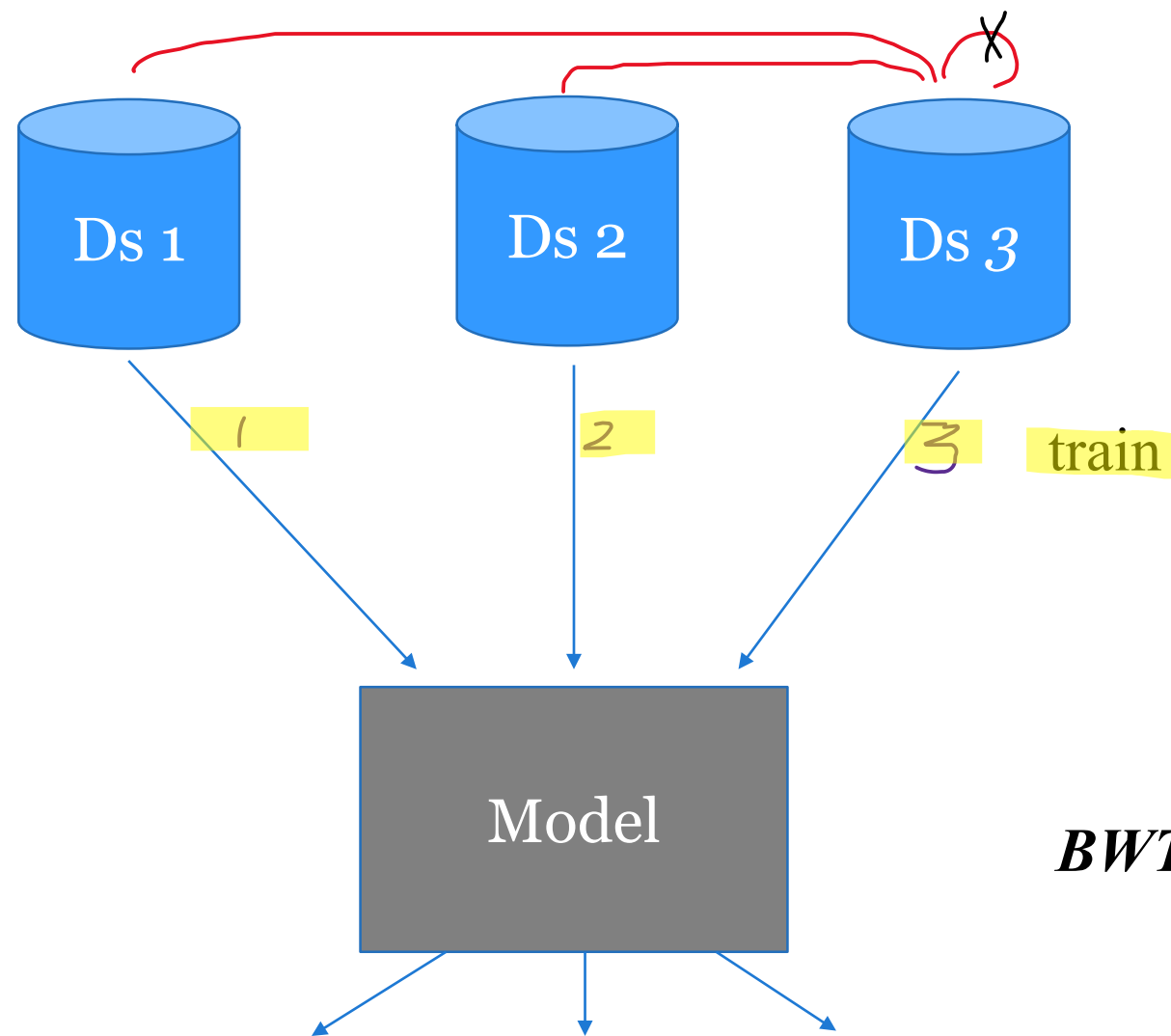
- 1) Freeze or protect old knowledge modules of models in the population
- 2) Extend architecture of models
- 3) Mutate only newly assigned modules or layers of models
- 4) Population = [current model, mutated variants, architecture-extended variants]

3. Select individual models that Perform well on new data while not degrading performance on old tasks

4. Integration

Best model becomes the new baseline. Maintain variants in the population.

5. Repeat as new tasks arrive.



$$BWT = [(70-90) + (80-85)]/2 = -12.5$$

Test accuracies:

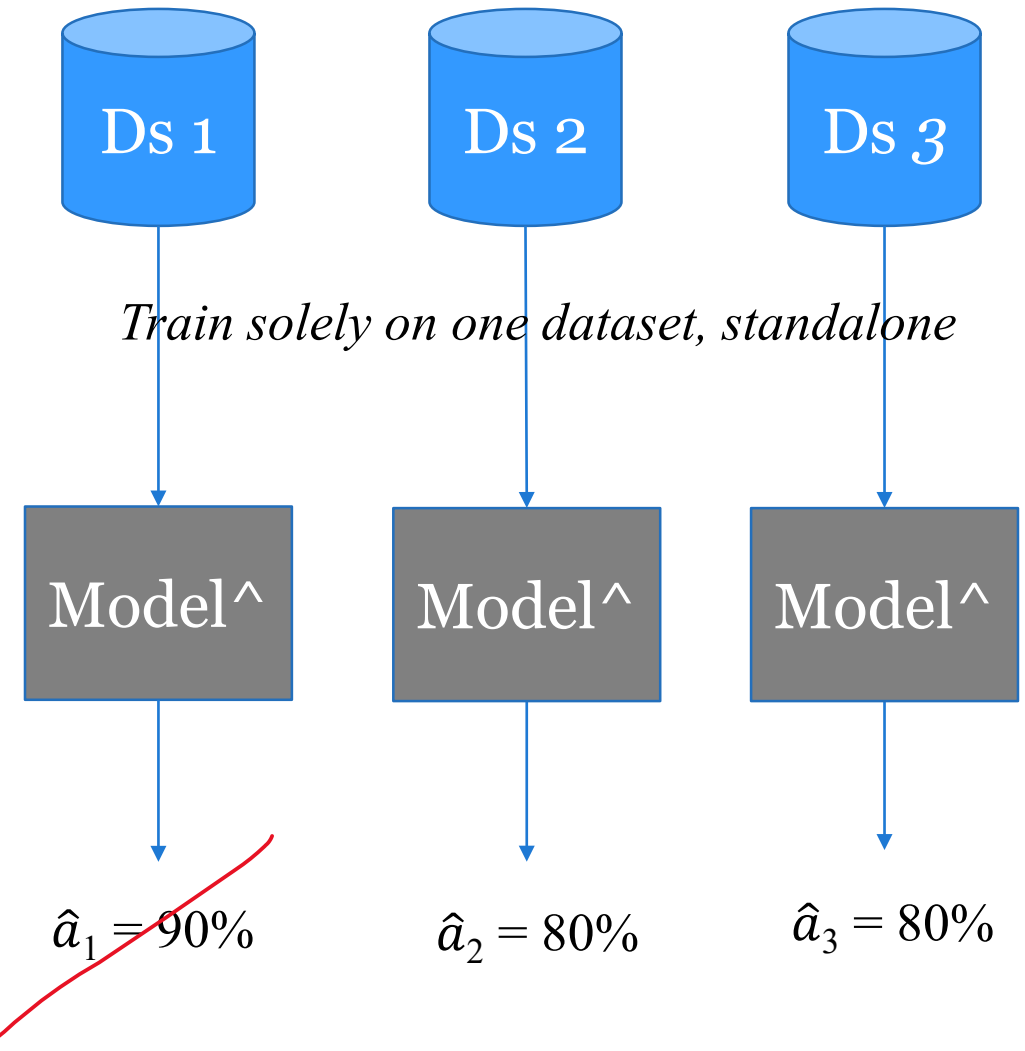
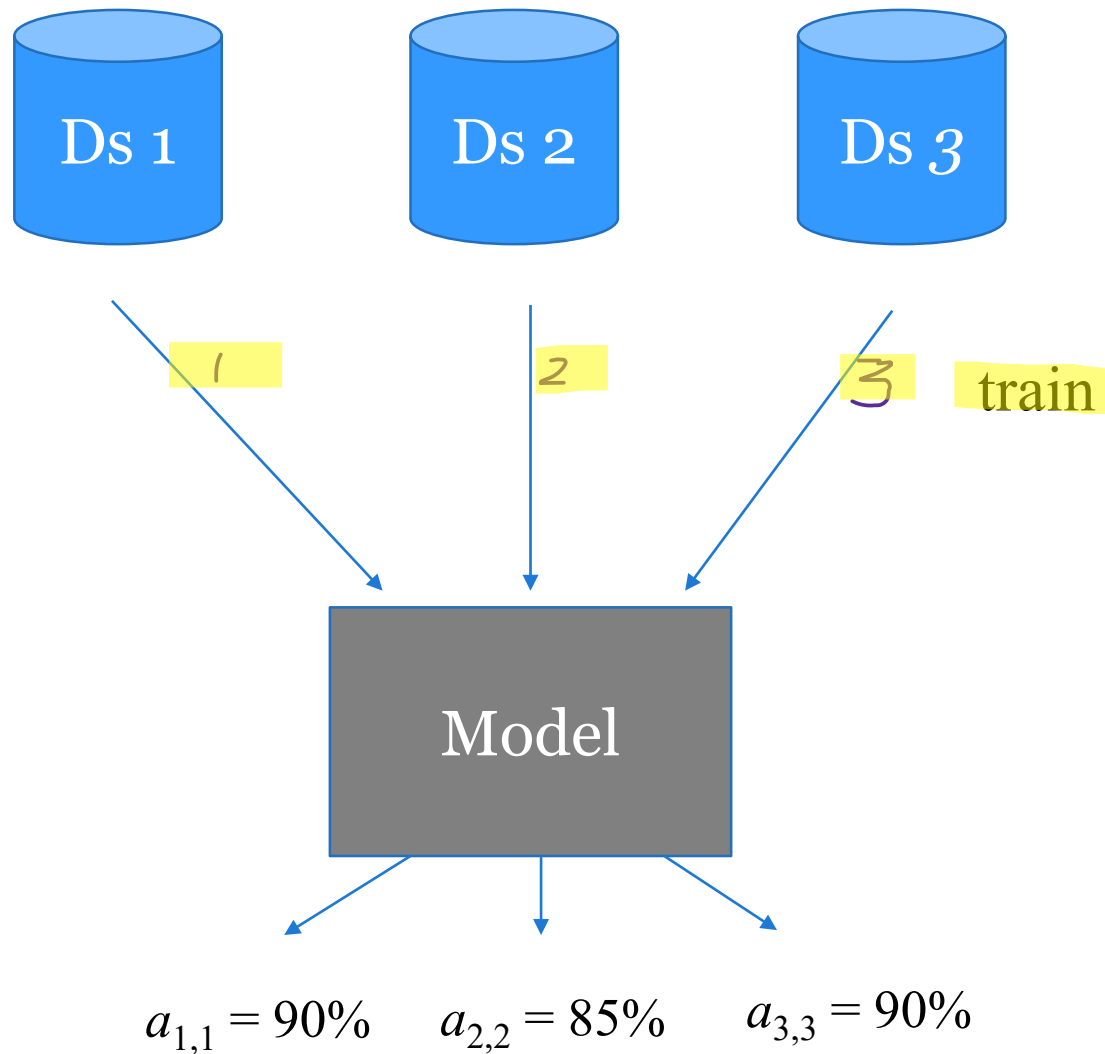
$$a_{1,1} = 90\% \quad a_{2,2} = 85\% \quad a_{3,3} = 90\%$$

$$a_{3,1} = 70\% \quad a_{3,2} = 80\% \quad a_{3,3} = 90\%$$

Forward Transfer

$$FWT_k = \frac{1}{k-1} \sum_{j=2}^k (a_{j,j} - \hat{a}_j)$$

- ACC and BWT measure performance within a continual setup. To quantify whether the continual training itself is beneficial for learning new tasks, use the forward transfer measure, FWT.
- $\hat{a}_j$ is the accuracy of a reference model trained solely on task j.
- Negative FWT values indicate that the sequential training on previous task has not led to a better-than-from-scratch performance.
- Ex: after training on some previous tasks, we reach a test accuracy of 90% on task j. A separate, randomly initialized model trained solely on task j's data reaches 80% accuracy. Then, the forward transfer would be +10, indicating that the continual training has been beneficial



$$FWT = [(a_{2,2} - \hat{a}_2) + (a_{3,3} - \hat{a}_3)] / 2 = [(85 - 80) + (90 - 80)] / 2 = 7.5$$

Backward Transfer2

$$BWT_k = \frac{1}{k-1} \sum_{j=1}^{k-1} (a_{k,j} - \hat{a}_j)$$

- Measures the **test-dataset performance differences** between solely (standalone) training on a task and after training on subsequent tasks.
- In most cases, this metric will be also negative which indicate forgetting